

12. Design a C program to simulate the concept of Dining-Philosophers problem

AIM

To write and execute a C program to simulate the **Dining Philosophers Problem** using threads and mutex locks.

C PROGRAM

```
#include <stdio.h>

#include <pthread.h>

#include <unistd.h>

#define N 5

pthread_mutex_t fork[N];

void *philosopher(void *arg)
{
    int id = *(int *)arg;
    int left = id;
    int right = (id + 1) % N;

    printf("Philosopher %d is thinking\n", id + 1);
    sleep(1);

    pthread_mutex_lock(&fork[left]);
    printf("Philosopher %d picked left fork\n", id + 1);

    pthread_mutex_lock(&fork[right]);
    printf("Philosopher %d picked right fork\n", id + 1);

    printf("Philosopher %d is eating\n", id + 1);
    sleep(2);
```

```

pthread_mutex_unlock(&fork[right]);
pthread_mutex_unlock(&fork[left]);

printf("Philosopher %d finished eating and released forks\n", id + 1);

return NULL;
}

int main()
{
    pthread_t thread[N];
    int id[N];

    for (int i = 0; i < N; i++)
        pthread_mutex_init(&fork[i], NULL);

    for (int i = 0; i < N; i++)
    {
        id[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(thread[i], NULL);

    for (int i = 0; i < N; i++)
        pthread_mutex_destroy(&fork[i]);

```

```
printf("\nAll philosophers have finished eating.\n");
```

```
return 0;
```

```
}
```

OUTPUT:

main.c	Run	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 #include <pthread.h> 4 #include <unistd.h> 5 6 #define N 5 7 8 pthread_mutex_t forks[N]; 9 10 void *philosopher(void *arg) 11 { 12 int id = *(int *)arg; 13 14 int left = id; 15 int right = (id + 1) % N; 16 17 printf("Philosopher %d is thinking...\n", id + 1); 18 sleep(1); 19 20 pthread_mutex_lock(&forks[left]); 21 printf("Philosopher %d picked up left fork.\n", id + 1); 22 23 pthread_mutex_lock(&forks[right]); 24 printf("Philosopher %d picked up right fork.\n", id + 1); 25 26 printf("Philosopher %d is eating...\n", id + 1); 27 sleep(2); 28 29 pthread_mutex_unlock(&forks[right]); 30 pthread_mutex_unlock(&forks[left]); 31 32 printf("Philosopher %d finished eating.\n", id + 1); 33 34 return NULL; 35 }</pre>		<pre>Philosopher 1 is thinking... Philosopher 3 is thinking... Philosopher 2 is thinking... Philosopher 4 is thinking... Philosopher 5 is thinking... Philosopher 1 picked up left fork. Philosopher 1 picked up right fork. Philosopher 1 is eating... Philosopher 3 picked up left fork. Philosopher 3 picked up right fork. Philosopher 3 is eating... Philosopher 5 picked up left fork. Philosopher 3 finished eating. Philosopher 2 picked up left fork. Philosopher 2 picked up right fork. Philosopher 2 is eating... Philosopher 5 picked up right fork. Philosopher 5 is eating... Philosopher 1 finished eating. Philosopher 4 picked up left fork. Philosopher 5 finished eating. Philosopher 4 picked up right fork. Philosopher 4 is eating... Philosopher 2 finished eating. Philosopher 4 finished eating. All philosophers have finished eating. === Code Execution Successful ===</pre>

RESULT

Thus, the C program was successfully executed to simulate the **Dining Philosophers Problem** using threads and mutex locks for synchronization.